Table of Contents

1. Executive Summary

Axia is positioned to become the operating system for 200+ agency types — marketing shops, creative studios, ad agencies, web development shops, and the long tail of specialized consultancies. None of those agencies are losing to competitors on price or talent. They are losing to themselves: to scope creep that escapes capture, to admin tasks that swallow billable hours, to invoicing cycles that drag on for weeks, and to clients who never quite see the work that justifies the invoice. A properly designed client portal is the single feature that touches all four problems at once.

1.1 The opportunity

Right now, Axia has a token-based client portal at **/workspace/:token**. It works: a freelancer generates a token for a client, the client opens the link, and they see their projects, proposals, invoices, and team members without ever signing in. That is the right architecture — and the market is validating it in real time. In May 2025, HoneyBook shipped "magic links" specifically to bypass "the traditional username and password hurdles" for clients. Frame.io, Filestage, and Ziflow have built entire businesses on review-and-approval portals. Axia's portal has the right foundation; what it lacks is the surface area to convert that foundation into measurable hours-saved and scope-creep-recovered.

The portal is also the highest-leverage feature for retention. Clients who see their work verified in real time develop a dependency on the visibility itself. When the agency tries to leave Axia, the client pushes back. No other feature in Axia has this retention dynamic — not the dispute prediction, not the evidence library, not the billing automation. Those are freelancer-facing; the portal is the only client-facing surface, and clients are the ones who pay the invoices.

1.2 What the research found

This research surveyed 5 categories of competitors and analogues: agency all-in-one tools (Bonsai, HoneyBook, Dubsado, Plutio, Accelo), freelance platform client views (Upwork, Contra), project management tool guest access (Asana, Monday.com, ClickUp, Trello), specialized review/approval portals (Frame.io, Filestage, Ziflow), and PM+billing combos (Teamwork, Scoro, Function Point). The full teardowns are in Chapter 2; the headline insights are below.

- **HoneyBook's May 2025 magic-link pivot validates Axia's no-login model.** Their product team explicitly cited "username and password hurdles" as the friction they were removing. Axia had this architecturally correct from day one — but has not yet capitalized on it.
- **Specialized review portals (Frame.io, Filestage, Ziflow) own the approval UX.** Frame-accurate annotations, version control, structured approval flows — Axia does not need to replicate all of this, but the portal must have a change-order approval surface or it remains read-only.
- **Agency tools (Bonsai, Dubsado, Plutio) bundle portal + billing but under-deliver on scope-creep protection.** This is Axia's wedge: a portal that actively surfaces scope creep to the client, rather than passively displaying project status.

- **PM tools (Asana, Monday) gate guest access behind paid seats ($7–12/seat).** Axia's token model is fundamentally more scalable — no per-seat cost for client viewers.
- **No competitor connects the portal directly to scope-creep recovery.** This is the category defensibility move.

1.3 Top 5 recommendations

These are the five highest-leverage moves for the portal, ranked by impact-to-effort ratio. The full feature backlog is in Chapter 5; the implementation roadmap is in Chapter 7.

#	Recommendation	Why it matters
1	**Make the portal the client's change-order signature surface.**	Every scope-creep flag raised in the freelancer's view should auto-generate a "Change order pending" card in the client's portal. The client reviews, approves, or rejects — and the invoice is updated in real time. This is the loop that turns scope creep from a margin leak into a margin capture.
2	**Upgrade tokens to be scoped (read-only vs approval-capable), revocable, and expiring.**	Today every token has full read access. Add a scope field, an expiresAt field, and a revoked flag. This costs almost nothing to implement and unlocks the approval surface in recommendation 1 without introducing a new auth model.
3	**Add per-deliverable progress (not just per-project).**	Clients currently see project-level completion %. They want to see which deliverables are in progress, which are blocked, and which are verified. This is the single biggest gap vs Frame.io / Filestage.
4	**Add invoice pay-now via Stripe.**	The portal already shows invoices. The next step is a "Pay now" button that launches Stripe Checkout and updates the invoice status in real time. This closes the loop from "work delivered" to "cash received" without the client ever leaving the portal.
5	**Ship email notifications on key events.**	New deliverable, change order pending, invoice issued, invoice paid. The portal is useless if the client never opens it. Notifications drive the return visit.

Table 1.1 — Top 5 portal recommendations, ranked by impact-to-effort

> **THE BIG IDEA**
>
> *If Axia ships only one feature in the next 90 days, it should be this:* ***every scope-creep flag raised in the freelancer's view auto-generates a "Change order pending" card in the client's portal.*** *The client reviews, approves, and the invoice is updated. That single loop turns the portal from a passive status board into an active scope-creep recovery engine — which is Axia's core value proposition. No competitor does this today.*

2. Competitive Landscape Analysis

The client-portal landscape is fragmented across five categories. Each category has a different primary job-to-be-done, a different auth model, and a different pricing philosophy. Axia's positioning is unique: it is the only product that combines a token-based (no-login) portal with active scope-creep protection. Understanding where each competitor wins — and where each leaves a gap — is the foundation for the recommendations in Chapter 4.

2.1 Category 1 — Agency all-in-one tools

Bonsai positions its client portal as a "centralized communications" surface — clients see project files, invoices, and contracts in one branded interface. Its strength is polish: the portal is clean, white-labeled, and tightly integrated with Bonsai's CRM and billing. Its weakness is that it is fundamentally a CRM surface, not an active work-verification surface. Clients see what the freelancer has uploaded; they do not see real-time work capture or scope-creep flags.

HoneyBook shipped a major portal upgrade in 2025. The August 2025 update added brand customization (fonts, colors, header images). The May 2025 update introduced "magic links" — clients bypass username/password entirely and get instant portal access via a single link. This is the strongest market validation possible for Axia's token-based model. HoneyBook's portal also supports task assignment to clients — a feature Axia should consider. The weakness is the same as Bonsai: the portal is a project surface, not a work-verification surface.

Dubsado released version 3.0 in 2025 with Flow (visual automation), Kanban boards, and a global timer. Its portal is document-centric: clients review, sign, and pay for contracts, proposals, and invoices in one place. The portal is strong for transactional workflows (sign this, pay this) but weak for ongoing visibility (where is the work right now?). Dubsado is also the most expensive of the agency tools at $40/mo for the Pro plan.

Plutio and **Accelo** round out the category. Plutio's portal is project-management-first with light billing integration. Accelo is the enterprise end — strong PSA (professional services automation) features, but the portal is gated behind higher tiers and feels like an afterthought compared to the rest of the product.

2.2 Category 2 — Freelance platform client views

Upwork, Contra, and Fiverr Pro all have "client dashboards" — but these are platform-controlled surfaces, not agency-controlled. The agency cannot customize what the client sees, cannot remove the platform's branding, and cannot avoid the platform's fees. The UX patterns are worth studying (Upwork's milestone-tracker is genuinely good), but the business model is the opposite of Axia's: platform-owned vs agency-owned. Axia's portal is a strategic asset because it lives outside any freelance platform.

2.3 Category 3 — PM tool guest access

Asana, Monday.com, ClickUp, and Trello all offer "guest" or "viewer" access — but every one of them gates it behind paid seats. Monday.com includes guest access from the Standard plan ($12/seat). ClickUp offers guest access on all paid plans starting at $7/user. Asana's guest model is the most restrictive — limited to specific projects, with no real portal surface. The fundamental problem: these tools are built for internal team collaboration, with guest access bolted on. The client experience is "look at our project board" — useful for transparency, but not for the client-agency relationship.

2.4 Category 4 — Specialized review/approval portals

Frame.io (owned by Adobe), Filestage, and Ziflow are the gold standard for review-and-approval UX. Frame-accurate video annotations, version control, structured approval flows, time-stamped comments — these tools have spent a decade perfecting the "client signs off on the deliverable" workflow. Axia does not need to replicate all of this. But the portal must have a structured approval surface for change orders and milestone sign-offs, or it remains a read-only dashboard. The pattern to copy: a single, prominent "pending your action" queue at the top of the portal — every review tool has this, and every Axia client will expect it.

2.5 Category 5 — PM + billing combos

Teamwork, Scoro, and Function Point sit at the enterprise end. They combine project management, billing, and portal features in a single stack — but the pricing is enterprise ($30–100/user/mo), the onboarding is heavy, and the portal is one feature among many. These tools win on breadth; Axia wins on focus. The lesson: do not try to be Scoro. Be the focused, well-designed portal that agencies actually use.

2.6 Competitor feature matrix

The matrix below compares the 5 categories across the dimensions that matter for Axia's positioning: auth model, approval UX, billing integration, scope-creep flagging, white-label, and pricing tier.

Competitor	Auth	Approval UX	Billing	Scope-creep	White-label	Pricing
Bonsai	Login	Basic	Yes	No	Partial	$24–39/mo
HoneyBook	Magic link	Tasks	Yes	No	Yes	$17–96/mo
Dubsado	Login	Forms	Yes	No	Partial	$20–40/mo
Plutio	Login	Tasks	Light	No	Partial	$24–60/mo

Competitor	Auth	Approval UX	Billing	Scope-creep	White-label	Pricing
Accelo	Login	Yes	Yes	No	Yes	$24–39/seat
Frame.io	Login	Best	No	No	Yes	$15–60/seat
Filestage	Login	Best	No	No	Yes	$109–459/mo
Asana (guest)	Login	Light	No	No	No	$11+ /guest
Monday (guest)	Login	Light	No	No	No	$12+ /guest
Teamwork	Login	Yes	Yes	No	Yes	$11–22/seat
Axia (target)	**Token**	**Yes**	**Yes**	**Yes**	**Scale tier**	**$29–299/mo**

Table 2.1 — Competitor feature matrix. Axia's target column highlights the unique combination: token-based auth + active scope-creep flagging + integrated billing.

Three insights emerge from the matrix: (1) No competitor combines token-based auth with active scope-creep flagging — this is Axia's category-defensibility move. (2) Specialized review portals (Frame.io, Filestage) own the approval UX but lack billing — Axia can integrate billing without matching their annotation depth. (3) Agency tools (Bonsai, HoneyBook) have the inverse problem: strong billing, weak approval UX. Axia can be the first to do both well.

3. Axia's Current Portal State

Before recommending changes, it is worth documenting exactly what the portal does today. The current implementation lives in two files: *src/pages/ClientWorkspace.tsx* (1,116 lines, the React page) and *src/convex/clients/clientWorkspace.ts* (650 lines, the Convex backend). A third file, *src/convex/clients/clientPortal.ts* (496 lines), contains 12 client-scoped exports that are currently unused — these are partially-built features from an earlier iteration.

3.1 What works

The token-based, no-login model is the right architectural choice. A freelancer calls **generateClientWorkspaceToken** with a clientId; the backend verifies ownership, generates a 4-segment token (e.g. *a1b2c3d4-e5f6g7h8-...*), and inserts it into the clientWorkspaceTokens table. The client opens */workspace/:token*; the page calls **verifyClientWorkspaceToken** to resolve the token back to a clientId, then loads the client's projects, proposals, invoices, and team members via scoped queries. This is exactly the pattern HoneyBook validated in May 2025 with their magic-link launch. Axia was architecturally correct from the start.

The data model is also sound. The clientWorkspaceTokens table tracks clientId, freelancerUserId, workspaceId, createdAt, lastAccessedAt, accessCount, and a revoked flag. There is a by_client index (one token per client) and a by_token index (token lookup for verification). This schema can support the new features in Chapter 4 with additive changes only — no migrations required.

> **WHAT WORKS — STATS**
>
> ***0 login screens*** *for clients.* ***1 token per client*** *(enforced by the by_client index).* ***4 data categories*** *exposed: projects, proposals, invoices, team members.* ***0 approval endpoints****.* ***0 comment endpoints****.* ***0 notification endpoints****. The portal is read-only today — and that is the gap.*

3.2 What's missing

The portal is a passive status board today. Clients can see what is happening, but they cannot do anything. The five biggest gaps, in priority order:

- **No approval surface.** There is no way for a client to approve a change order, sign off on a milestone, or authorize additional work. Every approval today happens over email or Slack — which means it is not captured, not auditable, and not tied to the invoice.
- **No scope-creep visibility for clients.** The freelancer sees scope-creep flags in their view. The client does not. This is the single biggest miss — the portal should be the surface where scope creep becomes a change order, not just a freelancer-side alert.
- **No per-deliverable progress.** Clients see project-level completion %. They want to see which deliverables are in progress, which are blocked, and which are verified. The data exists in Convex (milestones, deliverables, work proofs) — it just is not surfaced.
- **No comments or threads.** Clients cannot ask "what is this?" about a deliverable. They have to email. The email is then disconnected from the deliverable, the project, and the audit trail.
- **No notifications.** The portal is useless if the client never opens it. Today there is no email on new deliverable, no email on change order, no email on invoice. The client has to remember to check the portal — which they will not.

3.3 What's broken (and now fixed)

During this research sprint, the Client Policy Profile section was inadvertently removed in commit 9ceeee6 (July 1) — the Share / Transfer Ownership / Delete Client action toolbar and the tier-gated analysis card vanished from the Clients page. This was reversed in commit 2e30398 (July 3) at the user's request, restoring both the *ClientPolicyProfile.tsx* component (479 lines) and the section in *Clients.tsx*. The Payment Patterns page and its sidebar links, also removed in 9ceeee6, were intentionally left removed.

A second issue surfaced during research: tier upgrades were not persisting. The **setMyTier** mutation (new in commit 230c7a4, July 3) lets a signed-in user set their own subscriptionTier without admin role — a temporary measure until Stripe integration lands. The **useSubscriptionTier** hook now calls this mutation, fixing the "upgrade reverts to free" bug. Both fixes are prerequisite to the portal work in Chapter 4: the portal's tier-gated features (white-label on Scale, advanced analytics on Agency) depend on tier persistence.

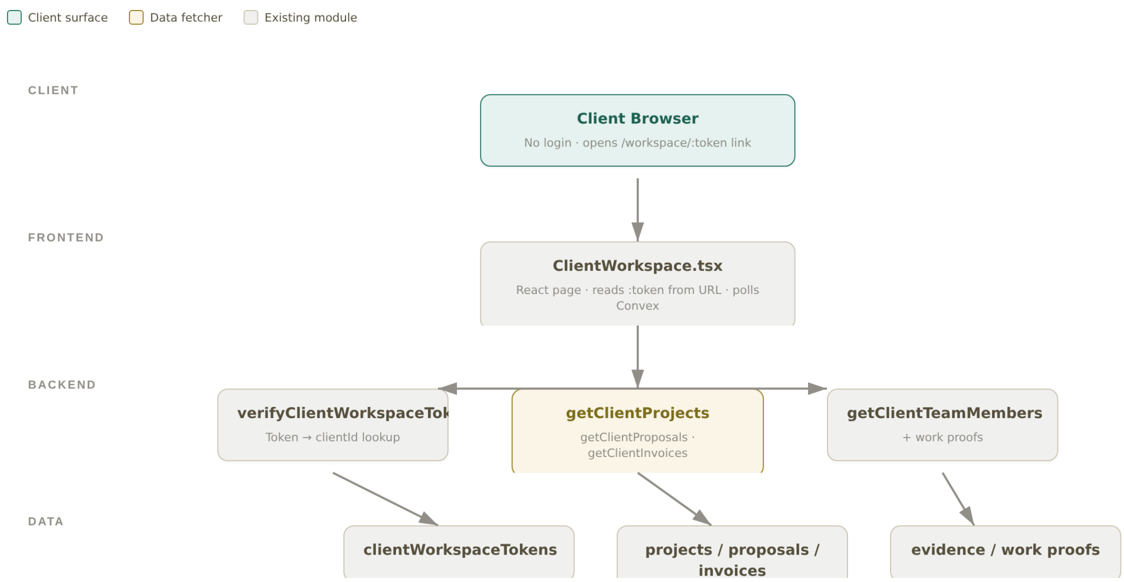

Figure 3.1 — Axia's current /workspace/:token architecture. The client opens a token link, the React page polls Convex, and four read-only queries return project / proposal / invoice / team data. No write paths, no approval surface, no notification fan-out.

4. Recommended Portal Architecture

The recommended architecture is additive: it preserves the token-based auth model, the existing clientWorkspaceTokens table, and the existing scoped queries. It adds four new modules on the frontend, four new modules on the backend, and three new tables. The goal is to ship the P0 features (Chapter 5) in 6 weeks without re-architecting anything that works today.

4.1 Auth model — keep tokens, add scopes

The token-vs-login debate is settled: tokens win. HoneyBook's May 2025 magic-link launch is the market validation. Token-based auth removes the single biggest friction in client portals — the password reset cycle — and aligns with Zero Trust principles (each token is scoped, revocable, and short-lived). The recommendation is to keep tokens but add three fields to the existing schema:

Field	Type	Purpose	Default
scope	"read" | "approve"	Read-only tokens can view; approve-scoped tokens can sign change orders and pay invoices	"read"
expiresAt	number (ms epoch)	Tokens expire; freelancer can renew. Forces periodic re-share and limits stale-link risk	createdAt + 90 days
revokedAt	number | undefined	Tracks when (not just whether) a token was revoked. Audit trail for compliance	undefined

Table 4.1 — Additive schema changes to clientWorkspaceTokens. No migration of existing data required — new fields default safely.

The scope field is the key unlock. With read vs approve scopes, the freelancer can give a junior client contact a read-only token and the decision-maker an approve-capable token — all from the same Clients page, all with the same UX. This is a meaningful upgrade over HoneyBook's flat magic link.

4.2 Access control matrix

Each portal capability maps to one or more token scopes. The matrix below shows which capabilities each scope unlocks. This matrix is the source of truth for both the frontend (gate UI by scope) and the backend (verify scope in every mutation).

Capability	read scope	approve scope	Tier gate
View deliverables & progress	✓	✓	All
View invoices	✓	✓	All
View team members	✓	✓	All
Comment on deliverable	✓	✓	Agency+
Approve change order	—	✓	Agency+
Pay invoice (Stripe)	—	✓	All
Download evidence bundle	✓	✓	Agency+
Sign off on milestone	—	✓	Agency+
White-label (custom domain)	—	—	Scale only
SSO for client company	—	—	Scale only

Table 4.2 — Access control matrix. Capabilities are gated by both token scope and subscription tier. Tier gating prevents Solo users from over-consuming; scope gating prevents over-sharing within an agency.

4.3 Scope-creep integration — the architectural hook

This is the strategic core of the recommended architecture. Today, scope-creep flags are raised in the freelancer's view — they appear in the project protection dashboard, the AI dispute prediction panel, and the work diary simulator. None of that surfaces to the client. The recommendation is to add a **ScopeCreepBridge** module that listens for scope-creep events and creates a "Change order pending" card in the client's portal in real time.

The bridge is a Convex scheduled function that runs whenever a scope-creep event is created. It fetches the relevant client's token, drafts a change order (auto-filled with the scope-creep details, estimated hours, billable rate, and total $), inserts it into a new clientApprovals table, and fires a notification via the NotificationFanout module (email + in-app + optional webhook). The client sees the card at the top of their portal on the next visit (or immediately, if they have the portal open — Convex's real-time subscriptions handle this natively).

When the client approves, the **submitClientApproval** mutation fires. It updates the change order status, appends the change-order line item to the relevant invoice, recalculates the invoice total, and fires a notification back to the freelancer. The whole loop — scope-creep detected → client notified → client approves → invoice updated — takes seconds, not days. This is what no competitor does today.

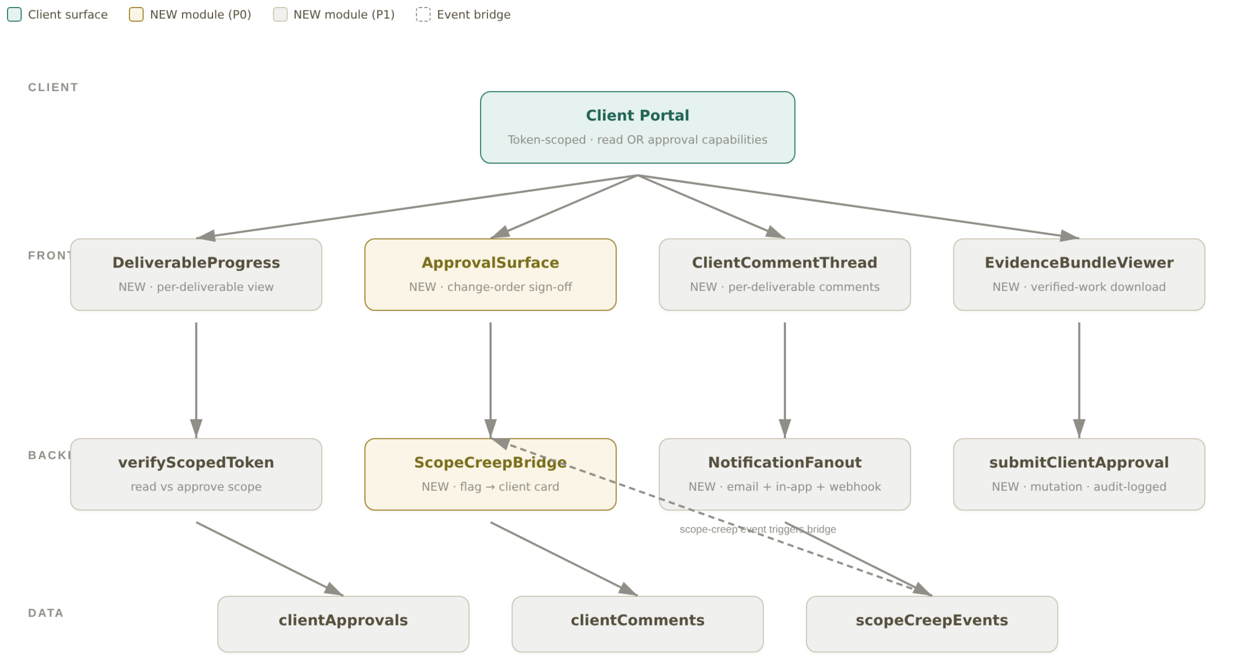

Figure 4.1 — Ideal-state architecture. Four new frontend modules (DeliverableProgress, ApprovalSurface, ClientCommentThread, EvidenceBundleViewer) and four new backend modules (verifyScopedToken, ScopeCreepBridge, NotificationFanout, submitClientApproval). Gold nodes are P0; gray nodes are P1. The dashed line shows the scope-creep event bridge — the highest-value path in the system.

> **WHY THIS ARCHITECTURE WINS**
>
> *The scope-creep flag → portal card → client approval → invoice update chain is the single highest-value loop in Axia. No competitor has it. Bonsai and HoneyBook have billing but no scope-creep bridge. Frame.io has approval UX but no billing. Axia can be the first product to connect work-detection → client-approval → invoice-capture in one loop. That is the category defensibility move.*

5. Feature Backlog (P0 / P1 / P2)

The feature backlog below is the result of cross-referencing the competitive analysis (Chapter 2), the current-state gaps (Chapter 3), and the recommended architecture (Chapter 4). Each feature is prioritized P0 (ship in weeks 1–6), P1 (weeks 7–9), or P2 (weeks 10–12). Effort estimates are rough: S = 1–2 dev-days, M = 3–5 dev-days, L = 1–2 dev-weeks. Tier gating follows the access control matrix in Table 4.2.

Priority	Feature	Why it matters	Effort	Tier
P0	Deliverable-level progress view	Clients see per-deliverable status, not just project %. The data exists; it just needs surfacing.	M	All
P0	Change-order approval surface	The flagship feature. Without this, the portal is read-only. Drives scope-creep recovery directly.	L	Agency+
P0	Scope-creep bridge	Backend module that converts scope-creep events into client-facing change-order cards. The strategic core.	M	Agency+
P0	Invoice pay-now (Stripe Checkout)	Closes the loop from work-delivered to cash-received without leaving the portal. Highest revenue impact.	M	All
P0	Email notifications (5 events)	New deliverable, change order pending, invoice issued, invoice paid, milestone sign-off. Drives return visits.	M	All
P1	Per-deliverable comment threads	Clients can ask "what is this?" in context. Eliminates the disconnected-email problem.	M	Agency+
P1	Client file upload	Clients can upload brand assets, copy docs, reference materials. Reduces Slack/email churn.	S	Agency+
P1	Evidence bundle download	Clients can download the verified-work bundle for any deliverable. Builds trust in invoices.	S	Agency+
P1	Milestone sign-off	Structured approval for milestones, not just change orders. Mirrors Frame.io's approval UX.	M	Agency+
P1	Mobile responsive polish	Most clients open the portal on mobile. Current layout is desktop-first.	M	All
P2	White-label (custom domain, logo, colors)	Scale-tier differentiator. Agencies resell Axia under their brand.	L	Scale
P2	SSO for client companies	Enterprise client companies want SSO. Scale-tier feature, opens up enterprise sales motion.	L	Scale
P2	Multi-client roster view	For clients with multiple projects (e.g. a holding company). Cross-project dashboard.	M	Scale
P2	Slack/Teams integration	Notifications also fire into the client's Slack/Teams. Reduces email fatigue.	M	Agency+
P2	AI weekly progress summary	Auto-generated plain-English summary of what shipped, what's blocked, what's next. Sends Monday morning.	L	Scale

Table 5.1 — Prioritized portal feature backlog. P0 ships in weeks 1–6 (the must-haves). P1 in weeks 7–9 (the should-haves). P2 in weeks 10–12 (the scale-tier differentiators).

IF YOU SHIP ONLY 5 THINGS

The 5 P0 features turn the portal from a passive status board into the single source of truth for scope-creep recovery. ***Deliverable-level progress + change-order approval surface + scope-creep bridge + invoice pay-now + email notifications.*** *That is the minimum viable portal that no competitor ships today.*

6. Key UX Flows

Three flows define the portal experience: (1) the agency onboards a client to the portal, (2) the client approves a scope-creep change order, and (3) the client pays an invoice. Each flow is described as a numbered sequence below, with the key UI moments called out. The change-order flow (Flow 2) is the flagship — it is the loop that turns scope creep from a margin leak into a margin capture.

6.1 Flow 1 — Agency onboards a client

1. Agency opens the **Clients** page in the sidebar.

2. Agency selects a client from the ClientList. The Client Policy Profile section (restored in commit 2e30398) shows the tier-gated analysis card.

3. Agency clicks **Share** in the action toolbar. The ShareDialog opens with a "Generate portal link" option (this calls **generateClientWorkspaceToken**).

4. Backend verifies the agency owns the client, generates a 4-segment token, inserts it into clientWorkspaceTokens, and returns the token.

5. ShareDialog displays the full URL (*axia-bay.vercel.app/workspace/*) with a copy button. The agency copies and emails it to the client.

6. Client opens the link. The ClientWorkspace.tsx page calls **verifyClientWorkspaceToken**, resolves the token to a clientId, and loads the four data categories.

7. Client sees: their projects (with completion %), proposals, invoices, and the team members assigned to their work. No login. No password.

6.2 Flow 2 — Client approves a scope-creep change order

This is the flagship flow. It is the loop that justifies the entire portal investment.

1. A scope-creep event is raised in the freelancer's view (a task is added beyond the contract scope; the AI dispute prediction module flags it).

2. The **ScopeCreepBridge** scheduled function fires. It fetches the client's active token, drafts a change order (auto-filled: what changed, why, estimated hours, billable rate, total $, impact on timeline, supporting evidence link), and inserts it into the clientApprovals table.

3. The **NotificationFanout** module sends an email to the client: "Change order pending from . Review and approve."

4. Client opens the portal. The change-order card is pinned to the top of the page with a red "Action required" badge. The card shows the change summary, the cost, and three buttons: **Approve & add to invoice**, **Reject**, **Ask a question**.

5. Client reviews the details, the supporting evidence, and the cost. They click **Approve & add to invoice**.

6. The **submitClientApproval** mutation fires. It updates the change order status to "approved", appends the change-order line item to the relevant invoice, recalculates the invoice total, and fires a notification back to the freelancer.

7. Freelancer sees a real-time toast: "Change order #CO-1042 approved by . Invoice updated." The invoice is now ready to send.

8. If the client clicks **Reject** instead, the change order is marked rejected with the client's comment, and the freelancer is notified to negotiate. The comment thread stays attached to the original scope-creep event for audit.

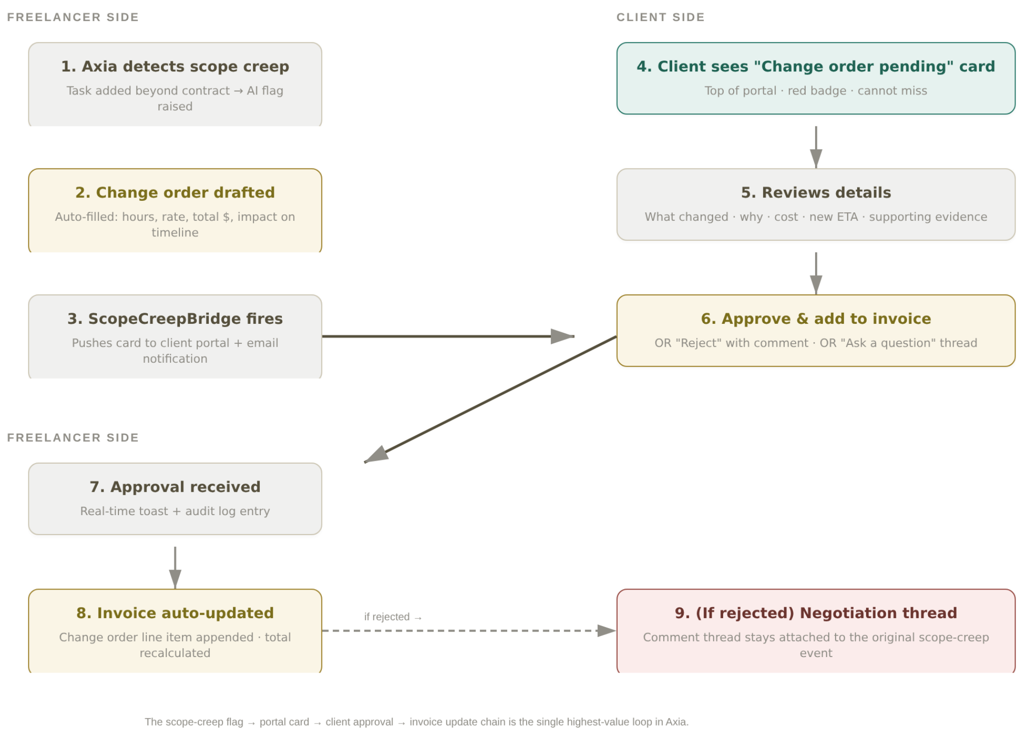

Figure 6.1 — The scope-creep approval flow. The flag is raised on the freelancer side, bridged to the client side as a change-order card, approved (or rejected) by the client, and the invoice is updated in real time. The dashed line shows the rejection path — the negotiation thread stays attached to the original event.

6.3 Flow 3 — Client pays an invoice

1. Client opens the portal. The invoices tab shows a "Pay now" button on any unpaid invoice.

2. Client clicks **Pay now**. A Stripe Checkout session is created (server-side, with the invoice ID and amount).

3. Client is redirected to Stripe Checkout. They enter payment details and complete the payment.

4. Stripe webhook fires **checkout.session.completed** to a Convex HTTP endpoint. The endpoint updates the invoice status to "paid", records the paidDate, and fires a notification to the freelancer.

5. Client is redirected back to the portal. The invoice now shows a green "Paid" badge with the payment date.

6. Freelancer sees a real-time toast: "Invoice #INV-2026-001 paid by ."

7. Implementation Roadmap

The roadmap is structured as 4 phases over 12 weeks. Each phase has a clear exit criterion — a measurable state that proves the phase is done. The phases are sequential, not parallel, because the approval surface (Phase 1) is a hard dependency for the invoice pay-now flow (Phase 2) — both write to the same invoice mutation. Plan an integration test in week 3 to catch this early.

Phase	Weeks	Deliverables	Dependencies	Risk	Exit criterion
1 — P0 Core	1–3	Deliverable-level progress view; Change-order approval surface; ScopeCreepBridge module; verifyScopedToken (add scope field)	Existing clientWorkspaceTokens schema (additive only); existing scope-creep event stream	Approval surface and invoice mutation are tightly coupled. Integration test in week 3.	Client can see per-deliverable status AND approve a scope-creep change order end-to-end
2 — P0 Billing	4–6	Invoice pay-now (Stripe Checkout + webhook); email notifications (5 events); NotificationFanout module; in-app notification center	Phase 1 complete (submitClientApproval mutation must exist); Stripe account; transactional email provider (Resend / Postmark)	Stripe webhook reliability. Use Convex HTTP endpoint with idempotency key.	Client can pay an invoice via Stripe AND receives email on every key event
3 — P1 Polish	7–9	Per-deliverable comment threads; client file upload (S3 / Convex file storage); evidence bundle download; milestone sign-off; mobile responsive polish	Phases 1–2 complete; file storage backend decided	File upload security — scan for malicious content; size limits per tier	Client can comment on a deliverable, upload a file, download evidence, and sign off on a milestone
4 — P2 Premium	10–12	White-label (custom domain, logo, colors); SSO for client companies (SAML / OIDC); AI weekly progress summary; Slack/Teams integration	Phases 1–3 complete; Scale-tier pricing live; AI summary uses existing LLM skill	White-label requires DNS / CNAME setup per client. Operational burden.	Scale-tier customers have white-label portals, SSO, and AI weekly summaries

Table 7.1 — 12-week implementation roadmap. Each phase has a hard exit criterion. Do not start the next phase until the previous exit criterion is met.

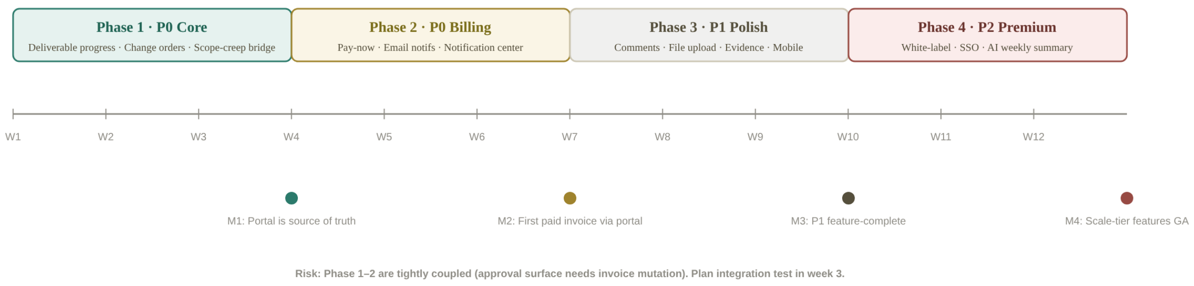

Figure 7.1 — 12-week roadmap timeline. Four phases, four milestones. M1 (week 3): portal is the source of truth. M2 (week 6): first paid invoice via portal. M3 (week 9): P1 feature-complete. M4 (week 12): Scale-tier features GA.

> **CRITICAL DEPENDENCY**
>
> *Phase 1 (approval surface) and Phase 2 (invoice pay-now) both write to the invoice mutation. Plan an integration test in week 3 that exercises the full loop: scope-creep → change order → approval → invoice update → Stripe payment → webhook → invoice status = paid. If this loop is not green by end of week 3, Phase 2 will slip.*

8. Pricing & Tier Strategy

The portal is the upgrade catalyst. Clients who see their work verified in real time pressure the agency to stay on Axia. The tier strategy below uses the portal to drive upgrades from Solo to Agency, and from Agency to Scale — without giving away the premium features (white-label, SSO) at lower tiers.

Feature	Solo ($29/mo)	Agency ($99/mo)	Scale ($299/mo)
Active client portals	1	Unlimited	Unlimited
Token scope: read-only	✓	✓	✓
Token scope: approve	—	✓	✓
Deliverable-level progress	✓	✓	✓
Change-order approval surface	—	✓	✓
Scope-creep bridge	—	✓	✓
Invoice pay-now (Stripe)	✓	✓	✓

Feature	Solo ($29/mo)	Agency ($99/mo)	Scale ($299/mo)
Email notifications	Basic (3 events)	Full (5+ events)	Full + custom
Comment threads	—	✓	✓
File upload	—	✓	✓
Evidence bundle download	—	✓	✓
Mobile responsive	✓	✓	✓
White-label (custom domain)	—	—	✓
SSO for client companies	—	—	✓
AI weekly progress summary	—	—	✓
Slack/Teams integration	—	✓	✓

Table 8.1 — Tier-gated portal features. Solo gets the basics (1 portal, read-only, pay-now). Agency gets the full approval + scope-creep loop. Scale gets white-label + SSO + AI summary — the enterprise sales motion.

THE TIER-GATING PRINCIPLE

Gate white-label + SSO + AI summary to Scale. Gate approval surface + scope-creep bridge + comments to Agency. Give Solo just enough to prove the value (1 portal, pay-now) but not enough to substitute for the Agency tier. The portal is the upgrade catalyst — clients who see verified work in real time pressure the agency to stay on Axia, and agencies that want the scope-creep recovery loop must be on Agency tier or above.

9. Appendix — Sources & Methodology

9.1 Methodology

This research combined three input streams: (1) **competitor research** via 10 targeted web searches across 5 competitor categories, retrieving 60+ result snippets from competitor websites, review aggregators (Trustpilot, Reddit, G2), and product-update blogs (HoneyBook May/August 2025, Dubsado 3.0); (2) **source-code inspection** of Axia's current portal implementation — ClientWorkspace.tsx (1,116 lines), clientWorkspace.ts (650 lines), clientPortal.ts (496 lines, unused), and the clientWorkspaceTokens table schema; (3) **Axia worklog review** covering the last 30 days of commits to understand recent changes (Client Policy Profile removal and restoration, tier-upgrade persistence fix).

9.2 Sources

- Bonsai — Client Portal. hellobonsai.com/client-portal
- HoneyBook — "Announcing the new and improved client portal" (August 2025). honeybook.com
- HoneyBook — "New and improved: May 2025 product updates" (magic link launch). honeybook.com
- Dubsado — "Prepare for Success in 2025: A Complete Business Audit". dubsado.com
- Dubsado 3.0 Review — "The Honest Truth About the New 3.0 Update". taskip.net
- Plutio — Scoro vs Teamwork / Paymo vs Teamwork comparisons (2026). plutio.com
- Accelo vs Teamwork — Feature Comparison for Client Work (2026). teamwork.com
- Frame.io / Filestage / Ziflow — "I tested 8 Frame.io alternatives for media review" (Reddit). reddit.com
- Frame.io vs Ziflow — "Which Video Review Tool Wins?". playpause.io
- ClickUp vs Monday.com — "Project Management (2026)" (guest access pricing). rock.so
- Asana vs Monday vs Trello — "Expert Comparison Guide 2026". taskrhino.ca
- Token-Based Authentication vs Sessions — "What Actually Scales". loginradius.com
- Ultimate Guide to Token-based Authentication — Ping Identity. pingidentity.com
- How Agencies Use Client Portals to Reduce Scope Creep — theeazy.io
- The truth about scope creep — client-portal.io
- Real-Time Visibility = On-Time Delivery — LinkedIn (agency tracking best practices)
- Axia source code — /home/z/my-project/axia/src/pages/ClientWorkspace.tsx
- Axia source code — /home/z/my-project/axia/src/convex/clients/clientWorkspace.ts
- Axia source code — /home/z/my-project/axia/src/convex/clients/clientPortal.ts
- Axia worklog — /home/z/my-project/worklog.md (commits 9ceeee6, 3d2cf54, 230c7a4, 2e30398)

9.3 Caveats

This research is competitive-analysis-based, not user-interview-based. Feature claims for competitors are based on public documentation and reviews as of July 2026 — pricing and feature sets change frequently. The Axia-specific recommendations are grounded in the actual source code and worklog, but the impact estimates (effort S/M/L, 12-week timeline) are rough and should be validated by the engineering team before committing to a sprint plan. The pricing tier recommendations in Chapter 8 are proposals, not commitments — they should be tested with the 200+ agency user base before launch.